

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	VIVETHA B	Reg. No.:	192424170
Date:		Duration:	60 minutes

Code of Conduct

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: VIVETHA B

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

PHISHING EMAIL DETECTION AND CYBER THREAT INTELLIGENCE SYSTEM

- **Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

Problem Statement

The **Phishing Email Detection and Cyber Threat Intelligence System** is designed to automatically identify phishing and suspicious emails. Phishing emails are fraudulent messages that attempt to steal sensitive information such as passwords, banking details, personal information, or login credentials. The proposed system uses **machine learning, email analysis, URL analysis, and cyber threat intelligence** to detect potential threats and provide a risk assessment.

Project Objectives

- **Detect phishing emails automatically** using machine-learning and rule-based techniques.
- **Classify emails** into categories such as **Legitimate, Suspicious, and Phishing**.
- **Analyze email content** including the sender, subject, body, keywords, and message structure.
- **Identify suspicious URLs and domains** present in emails.
- **Calculate a phishing risk score** based on multiple security indicators.
- **Integrate Cyber Threat Intelligence (CTI)** to obtain information about suspicious domains, IP addresses, and URLs.
- **Provide an easy-to-use dashboard** where users can submit emails and view analysis results.
- **Generate detailed security reports** showing detected threats, risk levels, and suspicious indicators.
- **Store email analysis results** in a database for future monitoring and investigation.

- **Use Git for version control** to maintain project history, support branching, collaboration, and safe code management.
- **Use Docker and Docker Compose** to create a consistent, portable, and easily deployable application environment.
- **Improve cybersecurity awareness and response** by helping users identify phishing attempts before they cause security or financial damage.

Expected Outcomes

The final system is expected to provide a **working web-based phishing detection platform** that analyzes an email, produces a phishing classification and risk score, identifies suspicious indicators, and presents cyber threat intelligence in a clear dashboard.

- **Project Structure Design**
 - Design and explain the folder structure of your project repository.

The **Phishing Email Detection and Cyber Threat Intelligence System** follows a modular project structure. The repository is divided into frontend, backend, machine learning, database, testing, documentation, and Docker-related components.

Project Repository Structure

phishing-email-cti/

```

|
├── backend/
|   ├── app/
|   |   ├── main.py
|   |   ├── routes/
|   |   |   ├── email.py
|   |   |   ├── threat.py
|   |   |   └── dashboard.py
|   |   └── services/
|   |       └── phishing_detector.py

```

- |— url_analyzer.py
 - |— threat_intelligence.py
 - |— models/
 - |— database_models.py
 - |— database.py
 - |— requirements.txt
 - |— Dockerfile

- |— frontend/
 - |— src/
 - |— components/
 - |— pages/
 - |— services/
 - |— App.jsx
 - |— package.json
 - |— Dockerfile

- |— ml/
 - |— dataset/
 - |— phishing_emails.csv
 - |— notebooks/
 - |— model_training.ipynb
 - |— train_model.py
 - |— phishing_model.pkl

- |— database/
 - |— init.sql

- |— tests/
 - |— test_email.py
 - |— test_detector.py
 - |— test_api.py

```
|— docs/
|   |— architecture.png
|   |— workflow.png
|   |— screenshots/
|
|— .gitignore
|— .env.example
|— README.md
|— docker-compose.yml
```

Explanation

1. **backend/**

Contains the server-side application. It processes email submissions, performs phishing detection, communicates with the database, and provides APIs.

2. **frontend/**

Contains the web interface of the project. Users can enter or upload emails and view phishing detection results, risk scores, and threat information.

3. **ml/**

Contains the machine-learning components, including the dataset, training notebook, training script, and trained phishing detection model.

4. **database/**

Contains SQL scripts required to create and initialize the project database.

5. **tests/**

Contains test cases for checking the email detection system, APIs, and other important functions.

6. docs/

Stores project documentation, system architecture diagrams, workflow diagrams, and screenshots required for the technical report.

7. .gitignore

Specifies files that should not be tracked by Git, such as `.env`, Python cache files, virtual environments, and `node_modules`.

8. .env.example

Contains sample environment variables required by the application without exposing actual passwords, API keys, or other secrets.

9. README.md

Provides information about the project, installation steps, execution instructions, technologies used, and usage instructions.

10. docker-compose.yml

Defines and manages the different containers required by the project, such as the frontend, backend, and database.

Why This Structure Is Suitable

This structure follows a **modular architecture**, where each major component has a separate responsibility. The frontend handles user interaction, the backend handles application logic, the ML folder manages phishing detection, the database stores results, and Docker manages deployment.

It also makes the project easier to:

- **Maintain** because related files are grouped together.
- **Develop collaboratively** because team members can work on different modules.
- **Test** individual components independently.

- **Use Git branches** for separate features.
 - **Containerize** individual services using Docker.
 - **Scale** the application by adding new services or modules.
 - **Debug** problems by isolating them to specific components.
 - **Deploy** the complete application using Docker Compose.
- Justify the purpose of each major folder and file included in the repository.

Folder / File	Purpose and Justification
backend/	Contains all server-side code. It processes user requests, performs phishing detection, handles APIs, and communicates with the database.
backend/app/	Contains the main backend application logic and keeps the source code organized into different modules.
main.py	Entry point of the backend application. It initializes the API server and connects the different application components.
routes/	Contains API routes for different functionalities such as email analysis, threat intelligence, and dashboard data.
email.py	Handles email submission and requests for phishing email analysis.
threat.py	Handles cyber threat intelligence requests and suspicious domain/URL information.

dashboard.py	Provides data required to display phishing statistics and security information on the dashboard.
services/	Contains the core processing logic of the application. Separating services makes the system easier to maintain and modify.
phishing_detector.py	Performs phishing email classification using the trained ML model and/or detection rules.
url_analyzer.py	Extracts URLs from emails and analyzes them for suspicious characteristics.
threat_intelligence.py	Handles threat-intelligence analysis of suspicious URLs, domains, or IP addresses.
models/	Contains database models that define how analysis results are represented and stored.
database_models.py	Defines tables/entities such as analyzed emails, risk scores, classifications, and detected indicators.
database.py	Establishes and manages the connection between the backend application and the database.
requirements.txt	Lists all Python libraries required to run the backend, making installation easier and consistent across systems.
backend/Dockerfile	Contains instructions for creating the Docker image for the backend application.
frontend/	Contains the user-facing web application where users submit emails and view analysis results.
frontend/src/	Contains the source code of the frontend application.
components/	Stores reusable interface elements such as cards, buttons, charts, tables, and result panels.

pages/	Contains major application pages such as Home, Email Analysis, Dashboard, and Threat Intelligence.
services/	Contains frontend code used to communicate with backend APIs.
App.jsx	Main React component that connects the different frontend pages and components.
package.json	Defines frontend dependencies, scripts, and project configuration.
frontend/Dockerfile	Provides instructions for building and running the frontend inside a Docker container.
ml/	Contains all machine-learning-related resources used for phishing email detection.
dataset/	Stores datasets used to train and evaluate the phishing detection model.
phishing_emails.csv	Contains examples of phishing and legitimate emails used for model training and testing.
notebooks/	Contains Jupyter notebooks used for data preprocessing, exploratory analysis, model training, and evaluation.
model_training.ipynb	Documents the complete machine-learning workflow, including preprocessing, training, testing, and performance evaluation.
train_model.py	Python script that can be used to train the phishing detection model automatically.
phishing_model.pkl	Stores the trained machine-learning model so that the backend can use it for predictions without retraining every time.
database/	Contains database-related configuration and initialization files.

init.sql	Creates the required database tables and initial database structure when the application is deployed.
tests/	Contains automated tests to verify that different components work correctly.
test_email.py	Tests email processing and validation functionality.
test_detector.py	Tests the phishing detection model and classification functionality.
test_api.py	Tests the backend API endpoints and their responses.
docs/	Stores technical documentation and supporting project materials.
architecture.png	Represents the overall architecture and communication between project components.
workflow.png	Shows the development, Git, Docker, or application workflow.
screenshots/	Stores screenshots of Git commands, Docker containers, application output, and testing results for the report.
.gitignore	Prevents unnecessary or sensitive files such as .env , virtual environments, cache files, and node_modules from being committed to Git.
.env.example	Provides a template showing required environment variables without exposing real passwords or API keys.
README.md	Provides project documentation, installation instructions, usage instructions, technologies used, and Git/Docker commands.
docker-compose.yml	Defines and manages multiple containers such as frontend, backend, and database, allowing the complete application to be started with a single command.

Overall Justification

The repository is divided into separate modules to achieve **separation of responsibilities**. The **frontend** manages user interaction, the **backend** handles application logic and APIs, the **ML module** performs phishing detection, the **database** stores analysis results, and the **tests** verify system functionality. The **Docker files** provide a consistent deployment environment, while **Git-related files** support version control and collaboration.

This structure makes the project **organized, maintainable, scalable, testable, and suitable for collaborative development**.

- **Git Repository Initialization**

- Create a Git repository for the project.

For the **Phishing Email Detection and Cyber Threat Intelligence System**, a Git repository is created to track source-code changes, maintain different versions of the project, support collaboration, and provide a complete development history.

Steps to Create the Git Repository

Step 1: Create the project folder

```
mkdir phishing-email-cti  
cd phishing-email-cti
```

Step 2: Initialize Git

```
git init
```

This creates a hidden **.git** directory and converts the project folder into a Git repository.

Step 3: Check the repository status

```
git status
```

This command displays the current state of the project and shows files that are not yet tracked.

Step 4: Add project files

```
git add .
```

This stages all project files so they can be included in the next commit.

Step 5: Create the first commit

```
git commit -m "Initial project structure"
```

This creates the first permanent version of the project in Git.

Step 6: Rename the default branch to `main`

```
git branch -M main
```

The `main` branch is used as the primary stable branch of the project.

Step 7: Connect the local repository to GitHub

After creating a GitHub repository named `phishing-email-cti`, connect it using:

```
git remote add origin https://github.com/USERNAME/phishing-email-cti.git
```

Step 8: Push the project to GitHub

```
git push -u origin main
```

This uploads the local project and its commit history to the GitHub repository.

Git Repository Workflow

Create Project

↓

```
git init
```

↓

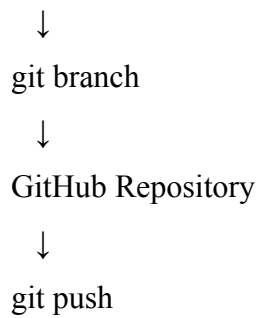
```
git status
```

↓

```
git add .
```

↓

```
git commit
```



Essential Git Files

File/Folder	Purpose
<code>.git/</code>	Stores Git's repository information, commits, branches, and configuration.
<code>.gitignore</code>	Prevents unnecessary or sensitive files from being tracked.
<code>README.md</code>	Provides project information and setup instructions.
<code>.env.example</code>	Shows required environment variables without exposing confidential values.

Example `.gitignore`

```
__pycache__/  
*.pyc  
.env  
venv/  
.venv/  
node_modules/  
*.log  
.DS_Store
```

Result

After initialization and the first push, the project will have a GitHub repository containing the complete project structure and an initial commit:

phishing-email-cti

```
|  
├── backend/  
├── frontend/  
├── ml/  
├── database/  
├── tests/  
├── docs/  
├── .gitignore  
├── .env.example  
├── README.md  
└── docker-compose.yml
```

Purpose: Git repository initialization provides a foundation for the later requirements of the assignment, including **branching, meaningful commits, merging, conflict resolution, synchronization, and commit-history analysis.**

- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

Steps Involved in Repository Initialization

1. Create the Project Directory

First, create a folder for the project and navigate into it.

```
mkdir phishing-email-cti
```

```
cd phishing-email-cti
```

This folder will contain the complete **Phishing Email Detection and Cyber Threat Intelligence System.**

2. Initialize the Git Repository

`git init`

This command initializes Git in the project folder and creates a hidden `.git` directory.

3. Check Repository Status

`git status`

This displays the current status of files and shows which files are untracked, modified, or staged.

4. Create a `.gitignore` File

Create a `.gitignore` file to prevent unnecessary and sensitive files from being tracked.

Example:

```
.env  
venv/  
__pycache__/  
*.pyc  
node_modules/  
*.log
```

5. Add Files to the Staging Area

`git add .`

This stages all project files for the first commit.

6. Create the Initial Commit

`git commit -m "Initial project structure"`

The commit permanently records the initial version of the project.

7. Create the Main Branch

`git branch -M main`

The `main` branch is used as the primary stable branch.

8. Connect to GitHub

After creating the GitHub repository:

```
git remote add origin https://github.com/USERNAME/phishing-email-cti.git
```

This connects the local Git repository to the remote GitHub repository.

9. Push the Repository

```
git push -u origin main
```

This uploads the project files and commit history to GitHub.

Essential Git Files and Their Purpose

Git File/Folder	Purpose
.git/	The most important Git directory. It stores commits, branches, configuration, and other Git metadata.
.gitignore	Specifies files and folders that Git should not track, such as passwords, virtual environments, cache files, and dependencies.
.git/config	Contains repository-specific Git configuration, such as repository settings and remote information.
README.md	Provides project information, installation instructions, usage instructions, technologies, and project objectives.
.env.example	Provides a template for environment variables such as database configuration and API keys without exposing actual secrets.
.git/HEAD	Indicates the currently active branch or reference.
.git/index	Stores information about files currently staged for the next commit.

.git/objec Stores Git objects such as commits, files, and directory information.

ts/

.git/refs/ Stores references to branches and tags.

Repository Initialization Workflow

Create Project Folder



git init



git status



Create .gitignore



git add .



git commit



Create main branch



Connect to GitHub



git push

Conclusion

Repository initialization establishes **Git-based version control** for the project. It allows every important change to the phishing detection system to be recorded, reviewed, restored, and shared. The **.gitignore**, **README.md**, **.env.example**, and **.git** directory are particularly important for maintaining a **secure, organized, and maintainable repository**.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).

For the **Phishing Email Detection and Cyber Threat Intelligence System**, a **Git Flow-based branching strategy** is suitable because the project contains multiple modules such as machine learning, frontend, backend, threat intelligence, and Docker deployment.

12 Key Points

- **Main Branch (`main`)**

The `main` branch contains the stable and production-ready version of the phishing detection system.

- **Development Branch (`develop`)**

The `develop` branch is used to combine completed features before they are released to the `main` branch.

- **Feature Branches (`feature/*`)**

Separate feature branches are created for individual functionalities, such as email detection, URL analysis, and dashboard development.

- **Email Detection Branch**

A branch such as `feature/email-detection` can be used to develop and test the phishing email classification module independently.

- **Threat Intelligence Branch**

A branch such as `feature/threat-intelligence` can be used to implement domain, URL, and IP threat analysis.

- **Dashboard Branch**

A branch such as `feature/dashboard` can be created for developing the frontend dashboard and visualization components.

- **Docker Branch**

A branch such as `feature/docker-deployment` can be used to develop Dockerfiles and the Docker Compose configuration.

- **Release Branch (`release/*`)**

A release branch such as `release/v1.0` is created when the project is ready for final testing and preparation for deployment.

- **Hotfix Branch (`hotfix/*`)**

A hotfix branch is used to quickly fix critical problems found in the production version, such as a security vulnerability or incorrect phishing classification.

- **Pull Requests and Code Review**

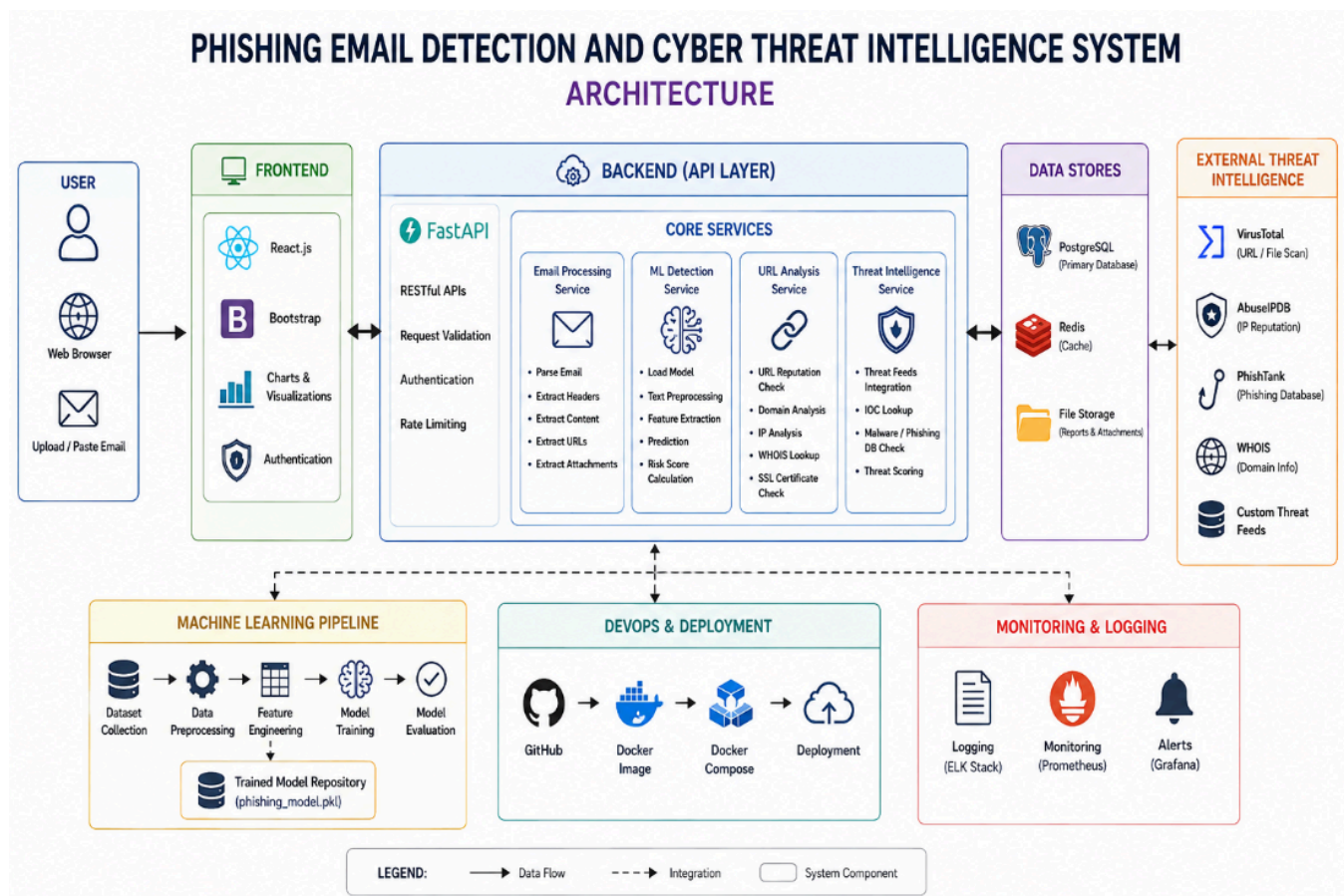
Feature branches should be merged through Pull Requests so that code can be reviewed and tested before being added to `develop`.

- **Controlled Merging**

Only tested and approved code should be merged from `develop` into `main`. This prevents unfinished features from reaching the stable version.

- **Improved Collaboration and Version Control**

This branching strategy allows multiple team members to work on different modules simultaneously while reducing conflicts and maintaining a clear project history.



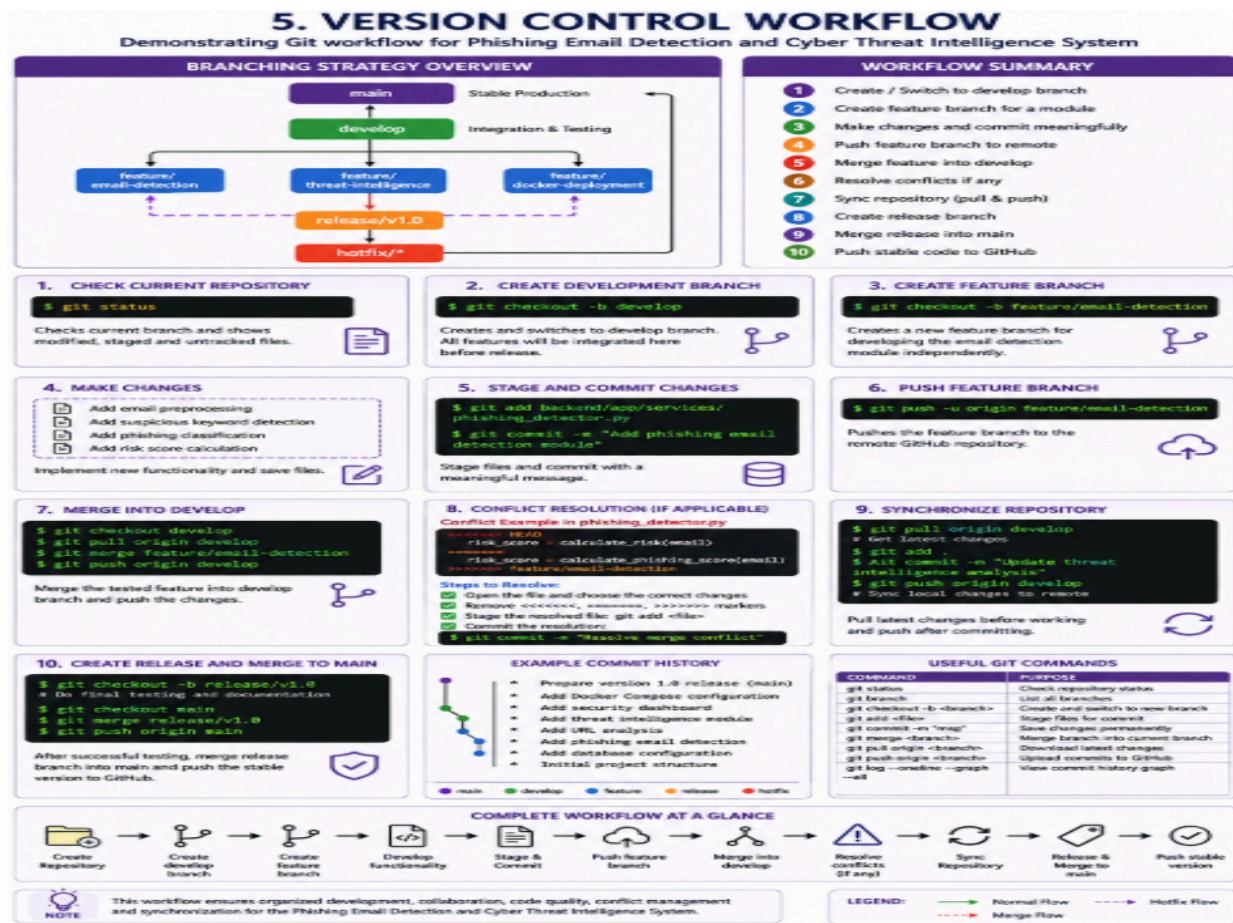
- Justify why the selected branching model is appropriate for the assigned project.
- **Supports Organized Project Development** – The Git Flow model divides the project into **main**, **develop**, **feature**, **release**, and **hotfix** branches. This provides a clear development process and prevents different types of work from becoming mixed together.
- **Enables Effective Team Collaboration** – Team members can independently work on different components such as phishing email detection, URL analysis, threat intelligence, frontend dashboard, and Docker deployment without directly affecting the stable application.
- **Maintains a Stable Main Branch** – The **main** branch contains only tested and reliable versions of the application. This is important for a cybersecurity project because unstable or incomplete security features should not be directly deployed.
- **Provides Separate Feature Development** – Each major functionality can be developed in its own feature branch, such as **feature/email-detection**, **feature/url-analysis**,

and **feature/threat-intelligence**. This makes development, testing, and debugging easier.

- **Reduces Code Conflicts** – Since developers work on separate branches, changes can be developed independently and merged carefully. This reduces the possibility of one developer accidentally overwriting another developer's work.
- **Improves Testing and Quality Control** – Completed features are first merged into the **develop** branch, where they can be integrated and tested together. Only after successful testing are they prepared for release.
- **Provides Better Security Management** – Security vulnerabilities, incorrect phishing classifications, and threat-intelligence issues can be developed and reviewed separately before being included in the production version.
- **Supports Controlled Software Releases** – A **release** branch can be created when the system is ready for final testing. Documentation, bug fixes, performance testing, and Docker deployment checks can be completed before merging into **main**.
- **Allows Fast Emergency Fixes** – If a serious security vulnerability or application failure occurs after deployment, a **hotfix** branch can be created to solve the problem quickly without disturbing ongoing development work.
- **Maintains a Clear Development History** – Git Flow provides a well-organized history of features, releases, and fixes. This makes it easier to identify changes, track contributors, restore previous versions, and maintain the **Phishing Email Detection and Cyber Threat Intelligence System** over time.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.



○ Explain the purpose of each Git operation performed.

- **git init** – Initializes the project directory as a Git repository and enables version control.
- **git status** – Checks the current state of the repository and identifies modified, staged, and untracked files.
- **git branch** – Displays the available branches and helps developers manage different versions of the project.
- **git checkout -b develop** – Creates the **develop** branch for integrating and testing project features.
- **git checkout -b feature/email-detection** – Creates a separate feature branch for developing the phishing email detection module without affecting the main code.
- **git add .** – Adds new and modified project files to the staging area before committing them.

- **git commit -m "Add phishing email detection module"** – Saves the completed changes in Git history with a meaningful commit message.
- **git push** – Uploads local commits and branches to the remote GitHub repository so that the project can be shared and backed up.
- **git pull** – Downloads the latest changes from GitHub and synchronizes the local repository with the remote repository.
- **git merge** – Combines changes from one branch into another, such as merging a completed feature into the **develop** branch.
- **Conflict Resolution** – Resolves differences when two branches contain conflicting changes to the same part of a file, ensuring that the correct code is retained.
- **git log** – Displays the project's commit history and helps track previous changes, features, and bug fixes.
- **git checkout -b release/v1.0** – Creates a release branch for final testing, documentation, and deployment preparation.
- **git merge release/v1.0** – Merges the tested release version into the stable **main** branch.
- **git push origin main** – Uploads the final stable version to GitHub and synchronizes the production

6. Commit History Analysis

- Present the commit history of the project.

The commit history records all important changes made during the development of the **Phishing Email Detection and Cyber Threat Intelligence System**. Each commit represents a specific development task and uses a meaningful message to clearly describe the changes.

Sample Commit History

- * 8f42c91 Prepare version 1.0 release
- * 7d31a84 Add Docker Compose deployment
- * 6a25f17 Add security monitoring dashboard
- * 5c18e63 Implement cyber threat intelligence module

- * 4b72d10 Add suspicious URL analysis
- * 3e64a92 Implement phishing email detection
- * 2d51b76 Add database configuration
- * 1c43a85 Create backend API
- * 0a29f71 Create frontend interface
- * 91b7c42 Initial project structure

Git Command to Display Commit History

```
git log --oneline --graph --all
```

This command displays the commits in a compact format along with the branch and merge structure.

Commit History Explanation

- **Initial project structure** – Creates the basic folders and files required for the project.
- **Create frontend interface** – Adds the user interface for submitting emails and viewing detection results.
- **Create backend API** – Implements the backend API responsible for processing user requests.
- **Add database configuration** – Adds database connectivity for storing email analysis results.
- **Implement phishing email detection** – Adds the machine-learning/rule-based phishing detection functionality.
- **Add suspicious URL analysis** – Implements URL extraction and analysis to identify potentially malicious links.
- **Implement cyber threat intelligence module** – Adds threat-intelligence functionality for analyzing suspicious domains, IP addresses, and other indicators.
- **Add security monitoring dashboard** – Adds charts and statistics for displaying phishing detection results and security information.
- **Add Docker Compose deployment** – Adds Docker configuration for running the application services together.
- **Prepare version 1.0 release** – Finalizes the tested application for the first stable release.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.

- **Clear and Descriptive** – Commit messages clearly explain what was changed, such as “*Implement phishing email detection*”, making the history easy to understand.
- **Short and Meaningful** – Messages are concise and focus on the main purpose of the change instead of providing unnecessary details.
- **Use Action Words** – Messages use action-oriented words such as *Add*, *Implement*, *Fix*, *Update*, and *Create* to clearly describe the performed operation.
- **Describe One Main Change** – Each commit focuses on a specific task, such as adding URL analysis or implementing the threat-intelligence module.
- **Easy to Track Features** – Meaningful messages help developers quickly identify when important features were introduced into the project.
- **Helps Debugging** – If a problem occurs, developers can examine commit messages and identify which change may have introduced the issue.
- **Improves Team Collaboration** – Team members can understand each other's work without opening every modified file.
- **Supports Code Maintenance** – Developers can easily locate previous changes when updating or modifying the phishing detection system.
- **Provides a Clear Development History** – Messages such as “*Add suspicious URL analysis*” and “*Add Docker Compose deployment*” show the project development process step by step.
- **Makes Rollback Easier** – Clear commit messages help developers identify the correct commit when they need to restore an earlier working version.

7. Docker Environment Design

- Explain why Docker is suitable for the assigned project.

Docker is suitable for the **Phishing Email Detection and Cyber Threat Intelligence System** because the project contains multiple components such as the frontend, backend, machine-learning model, and database. Docker packages these components into isolated containers and allows them to run consistently across different systems.

- **Consistent Environment** – Docker ensures that the application runs with the same software versions, dependencies, and configurations on different computers.
- **Easy Deployment** – The complete phishing detection system can be deployed quickly without manually installing Python, libraries, databases, and other dependencies.

- **Application Isolation** – Frontend, backend, ML services, and database can run in separate containers, preventing problems in one component from directly affecting others.
- **Dependency Management** – Docker packages all required libraries and dependencies with the application, reducing dependency conflicts.
- **Machine Learning Support** – The trained phishing detection model and its required Python libraries can be packaged inside the backend or ML container.
- **Database Containerization** – PostgreSQL or another database can run in its own container, providing an isolated and reproducible database environment.
- **Easy Team Collaboration** – All team members can use the same Docker configuration, reducing the common problem of “works on my machine.”
- **Portability** – Docker containers can run on Windows, Linux, macOS, cloud servers, and other supported environments with minimal changes.
- **Scalability** – Individual services can be scaled when required. For example, multiple backend containers can process a large number of email-analysis requests.
- **Easy Testing** – Developers can create isolated containers for testing the phishing detection API, database operations, and frontend without affecting their main environment.
- **Integration with Docker Compose** – Docker Compose can manage the frontend, backend, database, and other services together using a single configuration file.
- **Improved Maintainability** – Each service can be updated, restarted, or replaced independently, making the overall cybersecurity application easier to maintain.

- Identify the application components that require containerization.

For the **Phishing Email Detection and Cyber Threat Intelligence System**, the following components should be containerized to make the application portable, isolated, and easy to deploy.

- **Frontend Application**

The frontend contains the user interface where users submit emails and view phishing detection results, risk scores, and threat information. It should run in a separate Docker container.

- **Backend API Server**

The backend handles API requests, email processing, authentication, and communication between the frontend and other services. It should be containerized separately.

- **Phishing Detection / Machine Learning Service**

The ML component contains the trained phishing detection model and required Python libraries. Containerization ensures that the model works consistently with its required dependencies.

- **Email Processing Service**

This component extracts email headers, sender information, subject, body content, attachments, and other features. It can be included in the backend container or deployed as a separate service if the system becomes larger.

- **URL Analysis Service**

The URL analyzer extracts links from emails and checks them for suspicious characteristics. It can be containerized separately to provide isolation and easier scaling.

- **Cyber Threat Intelligence Service**

This service analyzes suspicious domains, IP addresses, URLs, and other indicators using threat-intelligence sources. Containerization makes the service independent from the main application.

- **PostgreSQL Database**

The database stores email analysis results, phishing classifications, risk scores, threat indicators, and scan history. It should run in its own database container with a persistent Docker volume.

- **Redis Cache**

Redis can be used to cache frequently requested threat-intelligence results and improve system performance. It can be deployed as a separate container if caching is required.

- **Background Task Worker**

A worker can process time-consuming operations such as bulk email analysis, threat-intelligence lookups, and ML predictions without blocking the main API. It can run in its own container.

- **Nginx / Reverse Proxy**

Nginx can be used as a reverse proxy to receive incoming requests and route them to the frontend and backend services. It can run in a separate container.

- **Monitoring and Logging Service**

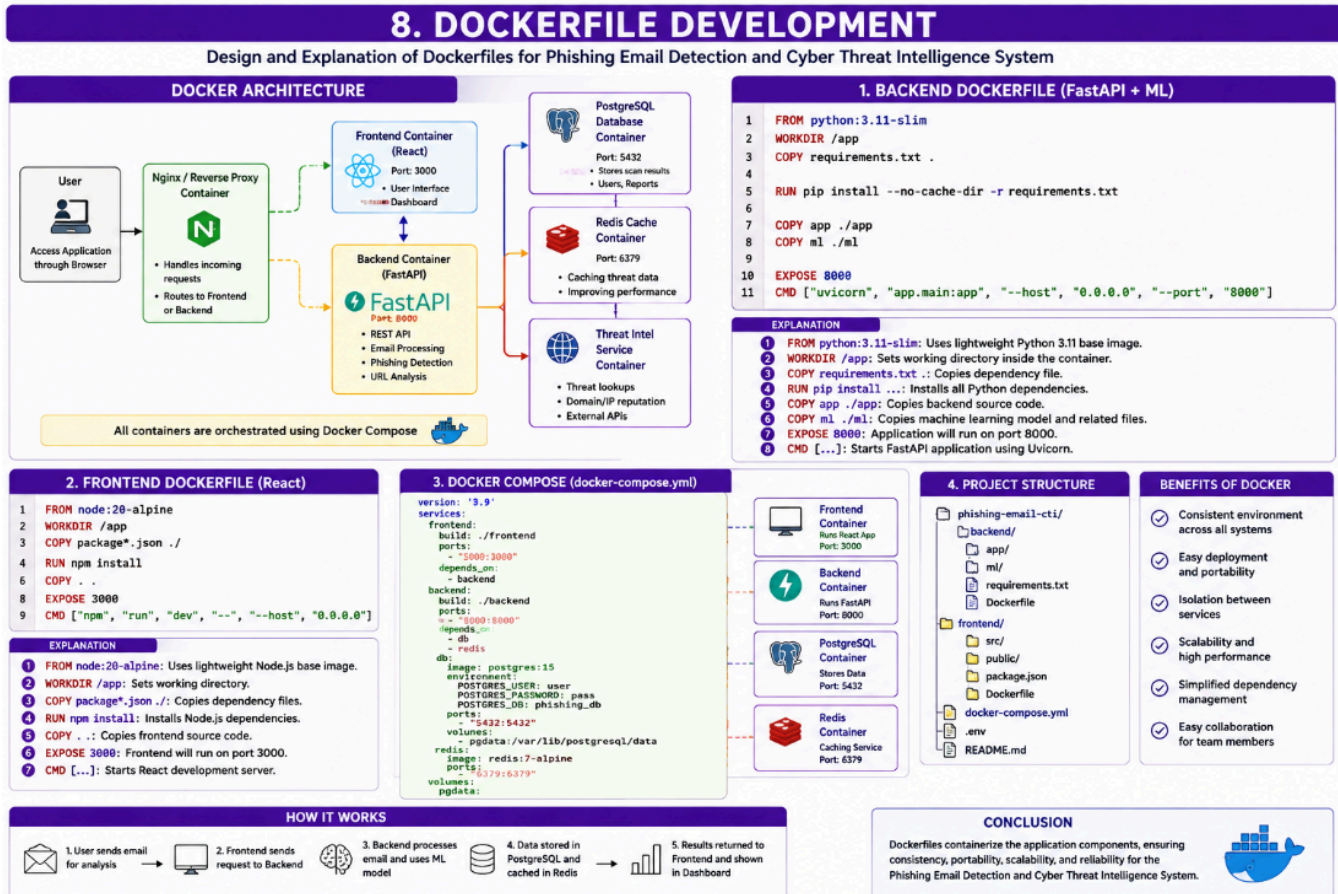
Monitoring components can collect application logs, container health information, and performance metrics. These services can be containerized when required.

- **Docker Compose Environment**

Docker Compose manages all the containers and allows the complete application to be started using a single command.

8. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.



- Describe the purpose of each instruction used in the Dockerfile.

For the **Phishing Email Detection and Cyber Threat Intelligence System**, the Dockerfile is used to create a consistent environment for running the backend application.

Backend Dockerfile

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app

COPY ml ./ml

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

Explanation of Each Instruction

1. FROM python:3.11-slim

Selects Python 3.11 as the base image. The **slim** version is lightweight and reduces the Docker image size.

2. WORKDIR /app

Creates **/app** as the working directory inside the container. All subsequent commands are executed from this directory.

3. COPY requirements.txt .

Copies the **requirements.txt** file from the project into the Docker container. This file contains all required Python packages.

4. RUN pip install --no-cache-dir -r requirements.txt

Installs the Python dependencies required by the application, such as FastAPI, Uvicorn, Pandas, and Scikit-learn.

5. COPY app ./app

Copies the backend application source code into the container so that the API and phishing detection functionality are available.

6. COPY ml ./ml

Copies the machine-learning files and trained phishing detection model into the container.

7. EXPOSE 8000

Documents that the backend application listens on port **8000** inside the container.

8. **CMD ["uvicorn", ...]**

Starts the FastAPI application using the Uvicorn server when the container starts.

Frontend Dockerfile

```
FROM node:20-alpine
```

```
WORKDIR /app
```

```
COPY package*.json ./
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

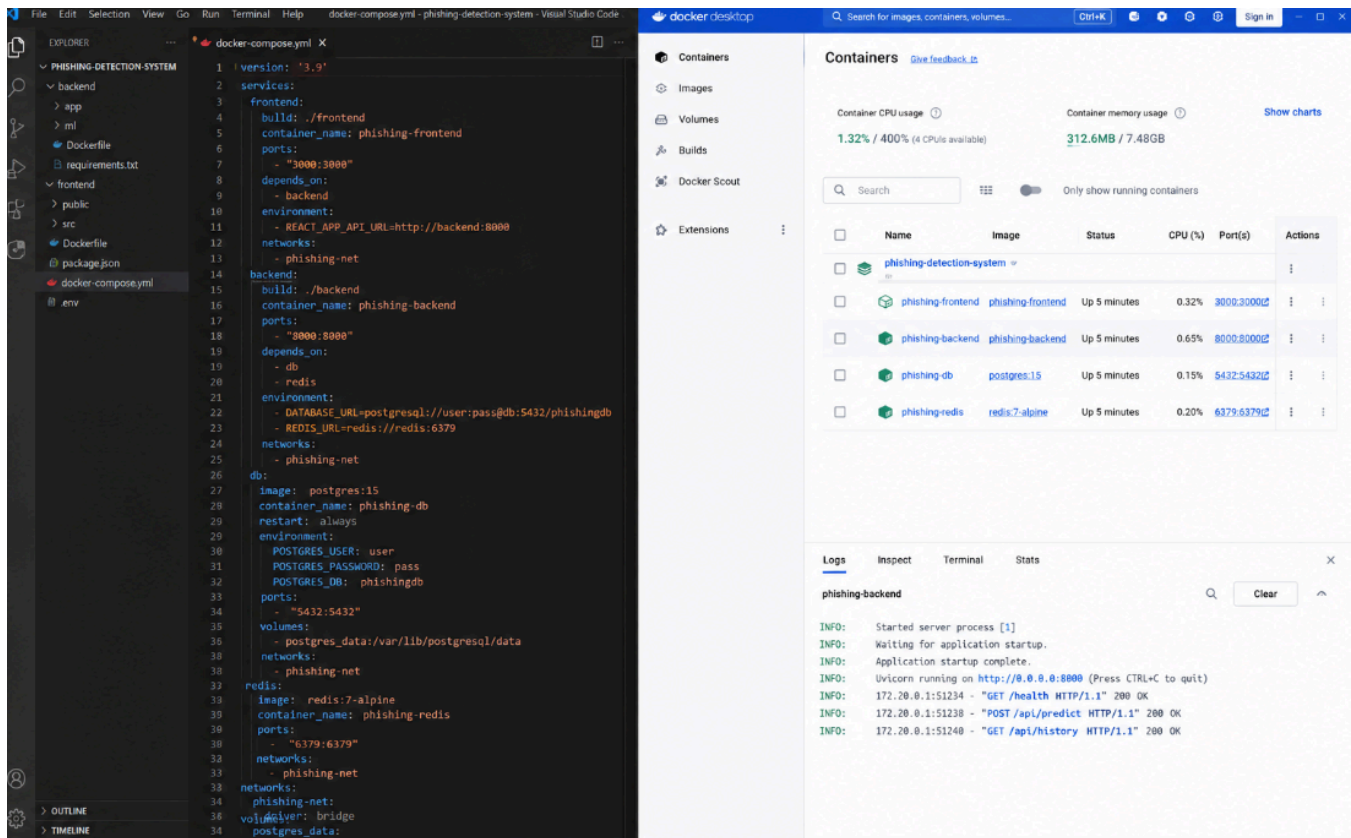
```
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

Explanation

1. **FROM node:20-alpine** – Provides a lightweight Node.js environment.
2. **WORKDIR /app** – Sets **/app** as the working directory.
3. **COPY package*.json .** – Copies the frontend dependency files.
4. **RUN npm install** – Installs the required Node.js packages.
5. **COPY . .** – Copies the frontend source code into the container.
6. **EXPOSE 3000** – Documents that the frontend application uses port **3000**.
7. **CMD [...]** – Starts the frontend development server when the container runs.

9. Docker Compose Design

- Develop a Docker Compose configuration for the project.



- Explain the services, networking, volumes, environment variables, and dependencies used.

Services

- **Frontend Service** – Runs the web interface and dashboard where users submit emails and view phishing detection results.
- **Backend Service** – Handles email processing, phishing detection, URL analysis, and API requests.
- **PostgreSQL Service** – Stores analyzed emails, classifications, risk scores, and threat intelligence results.
- **Redis Service** – Provides fast caching for frequently accessed threat-intelligence information.
- **Separate Containers** – Each service runs independently, making the application easier to manage, update, and scale.

Networking

- **Custom Network** – A Docker network such as `phishing-net` connects all project containers.
- **Service Communication** – Containers communicate with each other through service names instead of fixed IP addresses.
- **Frontend-to-Backend** – The frontend communicates with the backend through the backend API.
- **Backend-to-Database** – The backend connects to PostgreSQL using the database service name and port.
- **Backend-to-Redis** – The backend communicates with Redis through the Docker network for caching.

Volumes

- **Persistent Storage** – Docker volumes preserve important data even when containers are restarted.
- **Database Volume** – A `postgres_data` volume stores PostgreSQL database files.
- **Prevents Data Loss** – Removing or recreating a database container does not automatically remove its volume.
- **Improves Reliability** – Analysis history and stored threat information remain available after container restarts.
- **Easy Backup** – Persistent volumes make it easier to manage and back up application data.

Environment Variables

- **Database Configuration** – Variables define the PostgreSQL username, password, and database name.
- **Connection Information** – Backend variables specify the database and Redis connection URLs.
- **Security** – Sensitive information such as passwords and API keys can be kept outside the source code.
- **Flexible Configuration** – Different values can be used for development, testing, and production environments.
- **Easy Management** – Configuration can be changed without modifying the application source code.

Dependencies

- **Frontend Depends on Backend** – The frontend requires the backend API to process email-analysis requests.
- **Backend Depends on Database** – The backend requires PostgreSQL to store and retrieve analysis results.
- **Backend Depends on Redis** – Redis provides caching services required by the backend.
- **Startup Management** – `depends_on` defines the basic startup relationship between services.
- **Health Checks Recommended** – Health checks or retry mechanisms should be used to ensure dependent services are actually ready before the application uses them.

10. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.

The collage consists of 11 screenshots arranged in a grid-like fashion, showing the steps of container deployment and testing:

- 1. Build Docker Images:** A terminal window showing the execution of `docker compose build`, which builds the frontend and backend images.
- 2. Start Containers:** A terminal window showing the execution of `docker compose up -d`, which starts the containers.
- 3. Check Running Containers:** A terminal window showing the output of `docker compose ps`, listing the running containers.
- 4. Backend Logs:** A terminal window showing the logs of the backend service.
- 5. Docker Desktop (Running Containers):** A screenshot of the Docker Desktop interface showing the running containers.
- 6. FastAPI API Docs (http://localhost:8000/docs):** A screenshot of the FastAPI API documentation page.
- 7. Test Email Analysis Result:** A screenshot of the web application showing the analysis result of an email.
- 8. Test Database Connectivity:** A terminal window showing the execution of `docker compose exec db pg_isready` to test database connectivity.
- 9. Test Redis:** A terminal window showing the execution of `docker compose exec redis redis-cli ping` to test Redis connectivity.
- 10. Stop Containers:** A terminal window showing the execution of `docker compose down` to stop the containers.
- 11. Stop Containers:** A screenshot of the Docker Desktop interface showing the stopped containers.

- Explain the testing procedure and provide evidence that the application runs successfully.

- **Build the Docker Images** – Run `docker compose build` to verify that all Dockerfiles and application dependencies are configured correctly.
- **Start the Containers** – Run `docker compose up -d` to start the frontend, backend, PostgreSQL, and Redis services.
- **Check Container Status** – Use `docker compose ps` to verify that all containers are running without errors.
- **Test the Frontend** – Open `http://localhost:3000` and verify that the phishing detection dashboard loads successfully.
- **Test the Backend API** – Open `http://localhost:8000/docs` and verify that the FastAPI endpoints are available and responding.
- **Test Phishing Email Detection** – Submit a sample suspicious email and verify that the system identifies it as **Phishing** or **Legitimate** with an appropriate risk score.
- **Test URL Analysis** – Submit an email containing suspicious links and verify that the system detects and reports potentially malicious URLs.
- **Test Database Connectivity** – Run `docker compose exec db pg_isready` and verify that PostgreSQL returns “accepting connections.”
- **Test Redis Service** – Run `docker compose exec redis redis-cli ping` and verify that the response is “PONG.”
- **Provide Execution Evidence** – Capture screenshots of Docker containers, terminal commands, frontend dashboard, API documentation, and phishing-analysis results to demonstrate that the complete application runs successfully.

11. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

Git and Docker provide important benefits for the **Phishing Email Detection and Cyber Threat Intelligence System**. Git manages the project's source code and development history, while Docker provides a consistent environment for running and deploying the application.

Improved Collaboration

Git allows multiple team members to work on different branches simultaneously. For example, one member can develop the phishing detection model while another works on the dashboard or threat-intelligence module.

Better Version Management

Git records every important change through commits. Developers can view previous versions, compare changes, and restore an earlier working version if a problem occurs.

Easy Branch Management

Git branches allow features such as **email detection, URL analysis, threat intelligence, and Docker deployment** to be developed independently before being merged into the main project.

Portability

Docker packages the application and its dependencies into containers. The phishing detection system can therefore run consistently on different computers and operating systems without requiring the same manual configuration.

Consistent Development Environment

Docker ensures that all developers use the same versions of Python, Node.js, libraries, databases, and other dependencies. This reduces the "**works on my machine**" problem.

Faster Deployment

Docker allows the complete application to be started using Docker Compose commands. Frontend, backend, database, and other services can be deployed together without manually installing every dependency.

Scalability

Docker makes it easier to run multiple instances of services when the number of email-analysis requests increases. For example, additional backend containers can be deployed to handle more requests.

Better Maintainability

Git provides a clear history of changes, while Docker separates application components into containers. Developers can modify or update one component without unnecessarily changing the entire system.

Easier Testing and Recovery

Git allows developers to create branches for testing new features, while Docker provides isolated environments for testing the application. If a change causes a problem, the previous Git version can be restored and the Docker container can be rebuilt.

Reliable Deployment and Security

Git provides controlled access and version tracking, while Docker isolates application services. Together, they support safer deployment, controlled releases, easier debugging, and better maintenance of the cybersecurity application.

12. Challenges and Improvements

- Identify the challenges encountered during implementation.

During the development and deployment of the **Phishing Email Detection and Cyber Threat Intelligence System**, several technical and practical challenges may be encountered.

- **Phishing Detection Accuracy** – Designing a model that correctly distinguishes between legitimate and phishing emails can be difficult because attackers continuously change their techniques.
- **Dataset Quality** – Obtaining a sufficiently large, accurate, and balanced dataset containing both phishing and legitimate emails can be challenging.
- **Feature Extraction** – Extracting useful features such as suspicious keywords, URLs, sender information, email headers, and domain characteristics requires careful preprocessing.
- **Machine Learning Dependencies** – Libraries such as Pandas, Scikit-learn, and other ML packages may have compatibility or installation issues between different environments.
- **Docker Configuration** – Creating Dockerfiles with the correct base images, dependencies, working directories, ports, and startup commands can require significant configuration and testing.

- **Container Communication** – Configuring communication between the frontend, backend, PostgreSQL, and Redis containers can be challenging, especially when setting service names, ports, and networks.
- **Database Connectivity** – Establishing a reliable connection between the backend container and PostgreSQL requires correct credentials, connection strings, network configuration, and database readiness.
- **Environment Variables and Secrets** – Managing database passwords, API keys, and threat-intelligence credentials securely without accidentally committing them to Git is an important challenge.
- **Git Merge Conflicts** – When multiple developers modify the same files, merge conflicts may occur and require manual comparison and resolution.
- **Integration Testing** – Testing the complete system is more difficult than testing individual components because frontend, backend, ML model, database, and threat-intelligence services must work together correctly.
- **Performance and Scalability** – Processing a large number of emails and threat-intelligence requests can increase CPU, memory, and response-time requirements.
- **Deployment and Debugging Issues** – Errors in Docker containers, application logs, network configuration, or dependencies can make debugging difficult during deployment.

Overall Challenge

The main challenge was **integrating the machine-learning phishing detection system with the frontend, backend, database, threat-intelligence services, Git workflow, and Docker environment while maintaining accuracy, security, reliability, and portability.**

- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.
- ➔ **Implement CI/CD Pipeline** – Integrate GitHub Actions or another CI/CD tool to automatically build, test, and deploy the application whenever new code is pushed or merged.
- ➔ **Improve Branch Protection** – Protect the **main** branch by requiring Pull Requests, code reviews, and successful automated tests before merging changes.

- **Use Automated Testing** – Add unit tests, API tests, ML model tests, and integration tests that run automatically during Pull Requests.
- **Follow Standard Commit Conventions** – Use consistent commit messages such as `feat:`, `fix:`, `docs:`, and `test:` to make the project history easier to understand and maintain.
- **Use Version Tags** – Create Git tags such as `v1.0.0`, `v1.1.0`, and `v2.0.0` to clearly identify stable releases of the phishing detection system.
- **Improve Docker Image Size** – Use lightweight images such as Alpine or Slim images and multi-stage Docker builds to reduce image size and improve deployment speed.
- **Use Docker Health Checks** – Add health checks for the frontend, backend, PostgreSQL, and Redis services so that unhealthy containers can be detected automatically.
- **Secure Environment Variables** – Store passwords, API keys, and threat-intelligence credentials using secure secrets management instead of committing sensitive information to Git.
- **Add Container Security Scanning** – Use tools such as Docker Scout or other vulnerability scanners to identify vulnerable packages and base images before deployment.
- **Implement Container Monitoring** – Add monitoring and centralized logging to track CPU, memory, container health, API errors, and suspicious application activity.
- **Improve Scalability** – Deploy multiple backend containers when required and use a reverse proxy or load balancer to distribute phishing-analysis requests.
- **Automate Backup and Recovery** – Configure regular PostgreSQL backups and persistent Docker volumes so that important email-analysis and threat-intelligence data can be recovered after failures.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25